

# My Notebook Flutter Mobile Application

CEN306 Final Project Report

## 1. Project Purpose and Scope

My Notebook is a Flutter study notebook application designed to help students organize subjects, notes, tasks, reminders, search results, and study resources in one local-first app. The application supports different study modes such as Basic, Business, Engineering, and Medicine, then adjusts resources and dashboard content based on the selected field.

The app lets users register, log in, select a field, create subjects with visual notebook covers, write or draw notes, add sticky notes and emoji stickers, manage tasks and reminders, search stored content, use field-aware resources, and customize the app theme.

## 2. Main Functional Screens

| Item                | Details                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------|
| Authentication      | Login and sign-up screens register users and verify hashed passwords.                          |
| Mode Selection      | Field selection chooses the study context used throughout the app.                             |
| Dashboard           | Home screen summarizes subjects, notes, tasks, and reminders for the selected field.           |
| Subjects            | Users create, edit, delete, and open subjects with custom notebook covers.                     |
| Note Editor         | Users type, draw, erase, pick pen types, add sticky notes, add emoji stickers, and save notes. |
| Tasks and Reminders | Users track study tasks and dated reminders scoped to the current field and user.              |
| Search              | Users search subjects, notes, tasks, and reminders from one screen.                            |
| Resources           | Field-aware study templates and prompts can become subjects or tasks.                          |
| Theme               | Users select a theme color that updates the Material color scheme.                             |

## 3. Screen Evidence

The screenshots below show the current mobile web rendering used during verification. The same Flutter UI is shared with mobile builds.

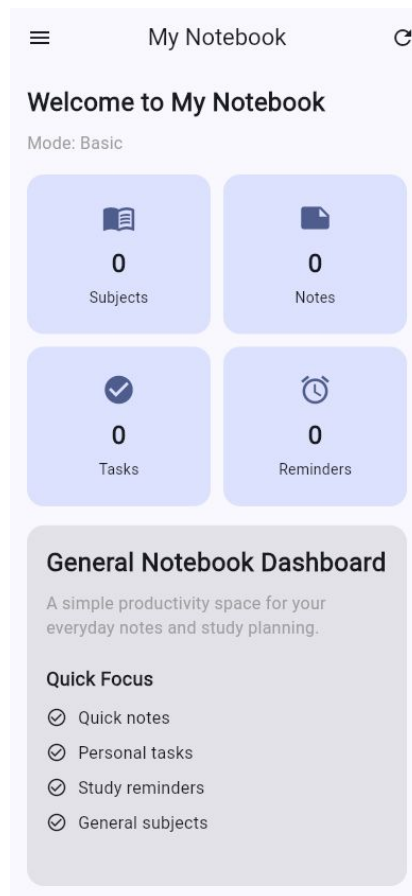


Figure 1. Mobile dashboard with responsive summary cards.

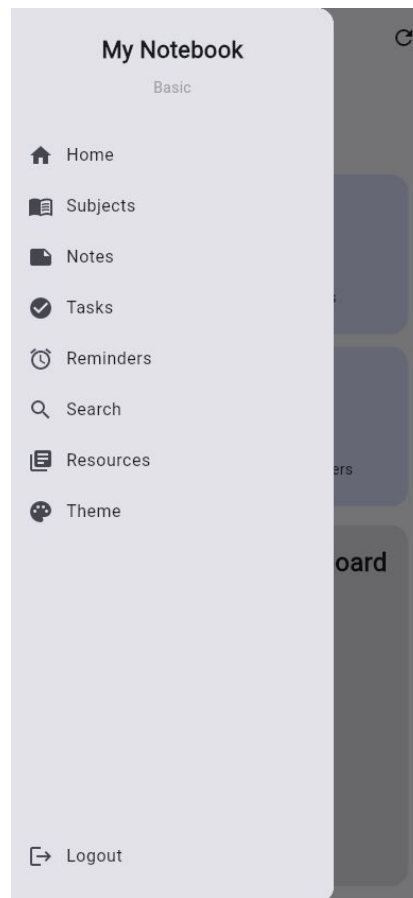


Figure 2. Drawer navigation on a compact mobile viewport.

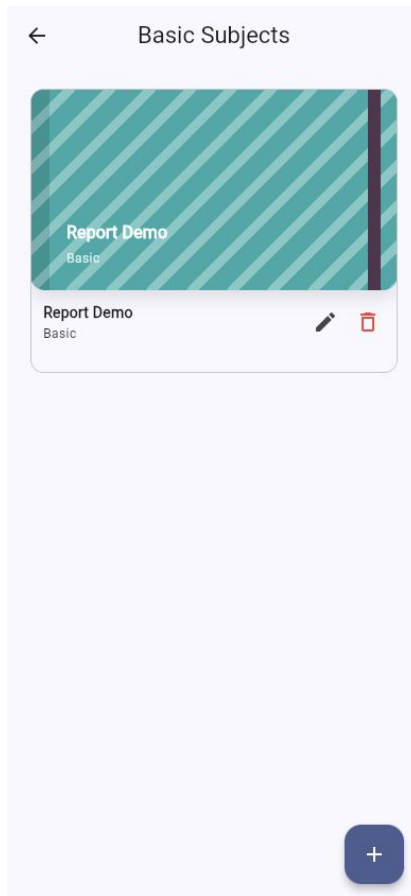


Figure 3. Subject list with visual notebook cover.

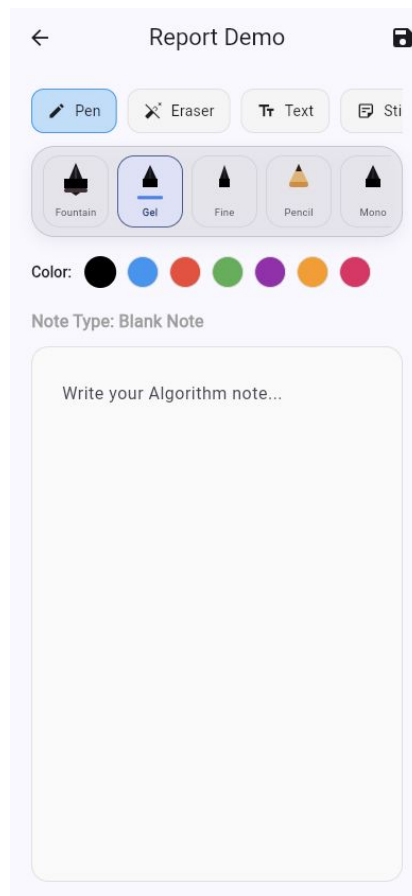


Figure 4. Note editor with pen-type tray and drawing tools.

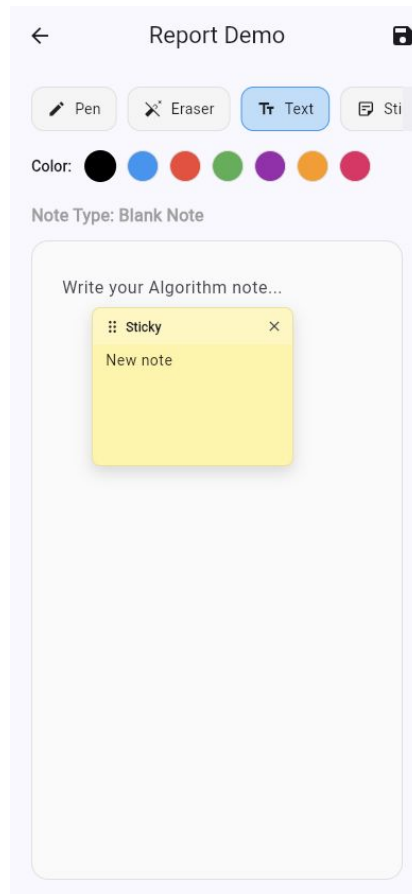


Figure 5. Sticky note attached to the notebook page.

## 4. Layered Architecture

The project is organized into separate UI, controller/business, repository, DAO, database, model, utility, and widget layers. This separation reduces direct coupling between Flutter screens and database implementation details.

- UI layer: lib/screens and lib/widgets contain visual layouts, navigation, forms, dialogs, and reusable UI elements.
- Controller layer: lib/controllers exposes actions used by screens.
- Repository layer: lib/repositories applies current-user scoping and app-facing data operations.
- DAO layer: lib/dao contains low-level table access for users, subjects, notes, tasks, and reminders.
- Database layer: lib/database defines the database adapter interface plus SQLite and web implementations.

## 5. SQLite, DAO, Repository, and Data Flow

On mobile and desktop targets the app uses sqflite through DBHelper. The web adapter mirrors the same AppDatabase interface with SharedPreferences so the app can still be demonstrated in Chrome. The core mobile data layer is SQLite.

| Item | Details |
|------|---------|
|------|---------|

|           |                                                                                                                                    |
|-----------|------------------------------------------------------------------------------------------------------------------------------------|
| users     | id, name, email, password. Stores account records with PBKDF2 password hashes.                                                     |
| subjects  | id, title, field, userEmail, coverColor, coverPattern. Stores user-scoped subject notebooks.                                       |
| notes     | id, subject, field, content, noteType, drawing, userEmail. Stores typed text, drawing JSON, sticky notes, stickers, and note type. |
| tasks     | id, title, isCompleted, field, userEmail. Stores user-scoped task checklist records.                                               |
| reminders | id, title, reminderDate, field, userEmail. Stores dated reminder records.                                                          |

Example note save flow:

- 1 SubjectNoteScreen collects text, strokes, sticky notes, stickers, note type, subject, and field.
- 2 NoteController receives the save request from the UI.
- 3 NoteRepository adds the current user email scope.
- 4 NoteDao performs the upsert operation against the notes table.
- 5 DBHelper supplies the platform database adapter.

## 6. State Management

The app uses StatefulWidget and setState for screens that hold changing data, including home counts, subjects, notes, tasks, reminders, search results, resource category selection, and the note editor. Theme color uses a ValueNotifier so MaterialApp can rebuild when the user selects a new theme.

- SubjectNoteScreen manages active tool, pen type, stroke list, eraser preview, sticky notes, stickers, selected template, and text editing state.
- HomeScreen updates dashboard counts after navigation and data changes.
- TasksScreen and RemindersScreen reload and update lists after CRUD actions.
- ThemeScreen uses AppThemeController.seedColor to update the app color scheme persistently.

## 7. Testing and Build Verification

| Item                 | Details                                                |
|----------------------|--------------------------------------------------------|
| dart format lib test | Passed with no files needing changes on the final run. |
| flutter analyze      | Passed with no issues found.                           |
| flutter test         | Passed all 6 tests.                                    |
| flutter build web    | Passed and generated build/web.                        |

|                           |                                                                                                               |
|---------------------------|---------------------------------------------------------------------------------------------------------------|
| flutter build apk --debug | Passed and generated app-debug.apk.                                                                           |
| Visual check              | Mobile web dashboard and drawer were checked at 390 x 844 viewport. Overflow was found, fixed, and rechecked. |

## 8. Development Challenges and Solutions

- Startup session handling now restores saved sessions and fields.
- Plain-text password storage was replaced with PBKDF2 SHA-256 hashing while supporting legacy upgrade on login.
- Field-only data separation was strengthened with userEmail scoping across subjects, notes, tasks, and reminders.
- The note editor now includes pen types, improved eraser behavior, sticky notes, and emoji stickers.
- HomeScreen now switches to drawer navigation on compact screens.
- Android builds currently pass but show a future Kotlin Gradle Plugin warning from shared\_preferences\_android.

## 9. Conclusion

The current project satisfies the main code-side expectations of the rubric: meaningful screens, working navigation, stateful UI behavior, SQLite-backed persistence, DAO/repository data flow, and successful build/test verification. Before final submission, student/course identity details can be added if required by the instructor.